Generics (tipos genéricos) em Java são um recurso introduzido na versão 5.0 que permite parametrizar tipos em classes, interfaces e métodos. Eles funcionam como "argumentos de tipo", permitindo que você crie códigos reutilizáveis, seguros e flexíveis para trabalhar com diferentes tipos de dados sem precisar reescrever a lógica. [1, 2, 3, 4, 5]

Principais Benefícios

- **Segurança de tipo em tempo de compilação (*Type Safety*):** O compilador verifica se você está usando o tipo correto, evitando erros apenas quando o programa está rodando. [1, 2]
- **Eliminação de *Casting* explícito:** Antes dos Generics, as coleções armazenavam objetos do tipo `Object`, o que exigia conversões manuais e arriscadas (`(String) lista.get(0)`). Com Generics, isso não é mais necessário. [1]
- **Reutilização de código:** Permite criar estruturas de dados genéricas — como a `ArrayList<E>` ou `HashMap<K, V>` — que funcionam com qualquer tipo de objeto. [1, 2]

Sintaxe Básica

Os parâmetros de tipo são representados por letras maiúsculas entre sinais de menor e maior (`<T>`). As convenções mais comuns são: [1, 2]

- `T` – Type (Tipo)
- `E` – Element (Elemento, muito usado em Coleções)
- `K` – Key (Chave)
- `V` – Value (Valor) [1]

Exemplo prático de uma classe genérica:

java

```java
// Criando uma caixa que pode guardar qualquer tipo de objeto
public class Caixa<T> {
    private T conteudo;

    public void guardar(T conteudo) {
        this.conteudo = conteudo;
    }

    public T pegar() {
        return conteudo;
    }
}
```

```
        // Usando a classe
        Caixa<String> caixaDeTexto = new Caixa<>();
        caixaDeTexto.guardar("Olá, Java!");
        String texto = caixaDeTexto.pegar(); // Não precisa de cast!
```

Use o código com cuidado.

Conceitos Avançados

- **Diamond Notation (`<>`):** Introduzido no Java 7, permite que você omita o tipo na instanciação do lado direito, deixando a inferência de tipo a cargo do compilador (ex: `List<String> lista = new ArrayList<>();`). [1]
- **Bounded Type Parameters (Delimitação de Tipos):** Você pode restringir os tipos aceitos usando a palavra-chave `extends`. Exemplo: `<T extends Number>` aceita apenas `Integer`, `Double`, etc. [1, 2]
- **Wildcards (Curingas com `?`):** Utilizados quando você quer representar um tipo desconhecido.
  - `?` – Representa qualquer tipo.
  - `? extends T` – Aceita o tipo `T` ou subclasses dele.
  - `? super T` – Aceita o tipo `T` ou superclasses dele. [1, 2, 3]
- **Type Erasure (Apagamento de Tipo):** Por baixo dos panos, o Java remove as informações de tipo genérico durante a compilação para manter a compatibilidade com versões antigas da JVM (um `List<String>` vira apenas um `List` em bytecode). [1, 2]

Para mais detalhes sobre a especificação oficial, consulte a Documentação de Generics da Oracle ou veja boas práticas no artigo do Medium sobre Java Generics. [1, 2]